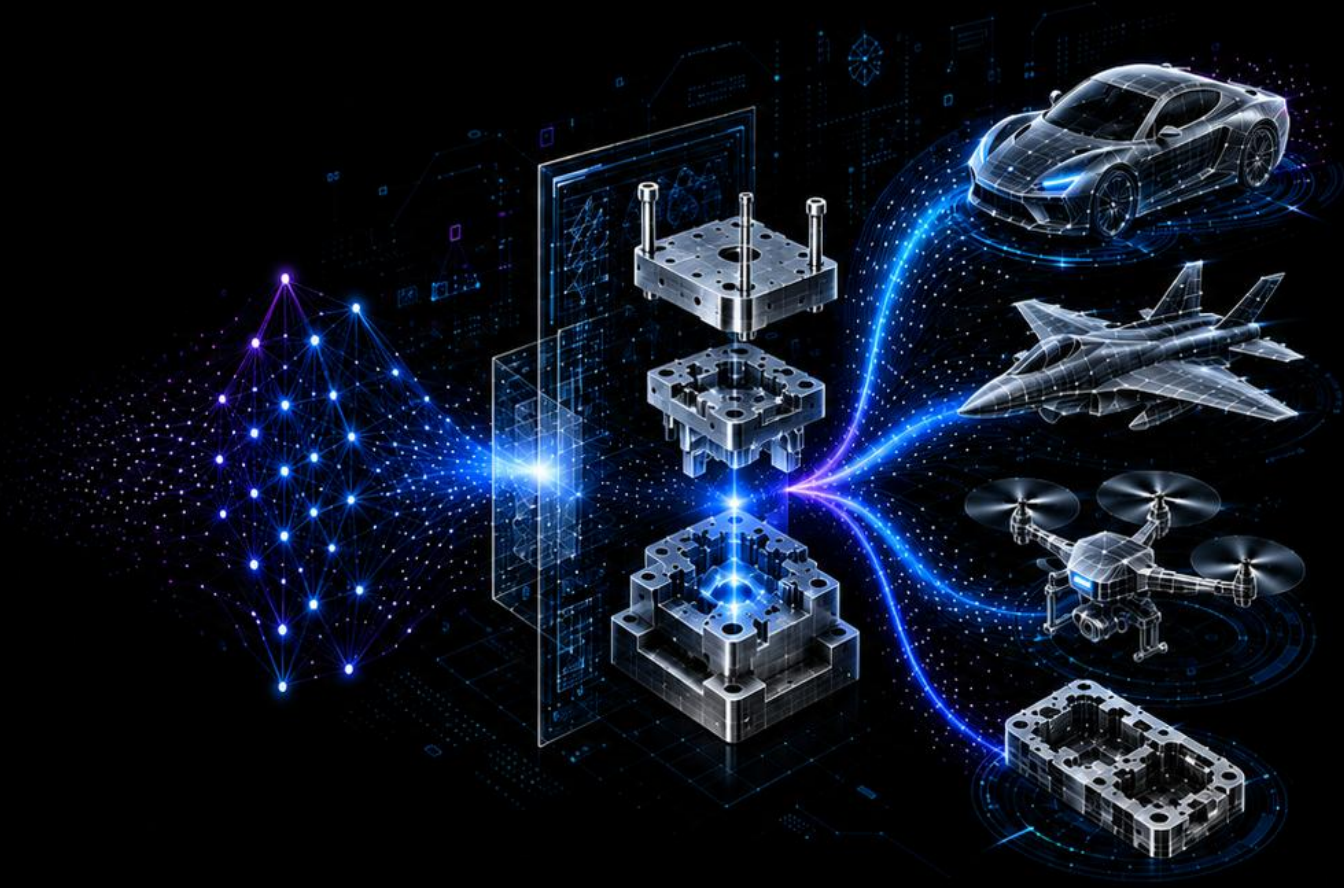
26.09.22 Meeting

Graph-RAG & KG-RAG 조사 및 Original Chunk에 Contextual Retrieval 적용 및 코드 구현

신동우

(지도교수: 양선웅)

한양대학교 ERICA 기계공학과



Intelligent Design for Engineering Applications Lab

Meeting List

List

- Graph-RAG, 논문 조사

- Knowledge Graph based Retrieval Augmented Generation (KG-RAG) Framework, 논문 조사

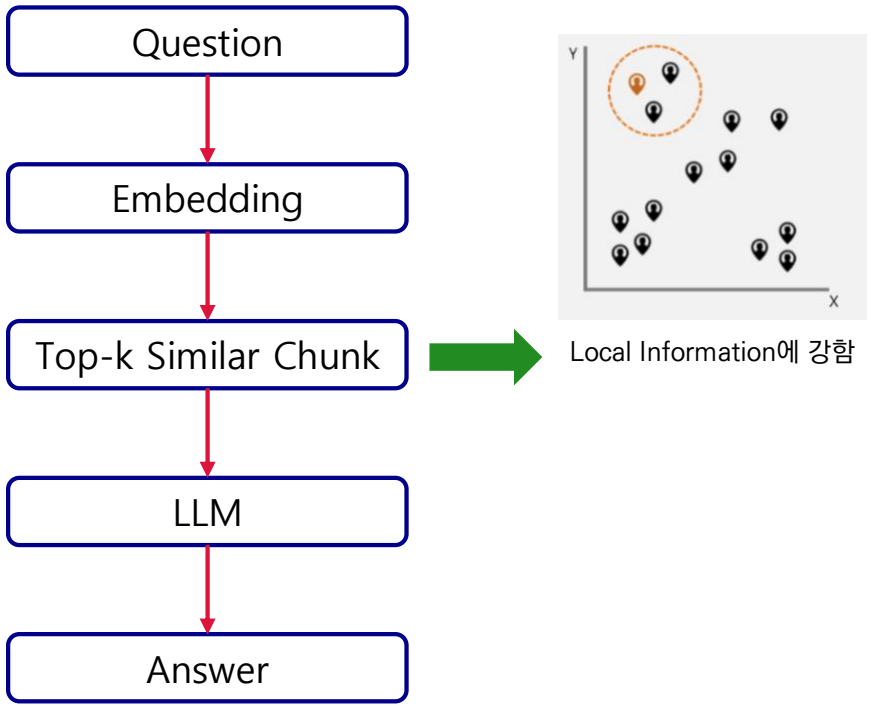
- Original Chunking + Contextual Retrieval Preprocessing, Chunking과 관련된 5개의 논문에서 말하는 방법을 참고하여 코드 구현

Why Graph-RAG

- "From Local to Global: A Graph-RAG Approach to Query-Focused Summarization", 2024

Limitation of Vector RAG

❖



Example

❖ Local Question이면?

"NeoChip은 언제 Quantum System에 인수되었는가?"
➔ 인수된 정보 부분만 찾아서 답변하면 된다

❖ Global Question이면?

"전체 문서에서 가장 중요한 주제는 무엇인가?" 또는
"전체 데이터에서 반복적으로 나타나는 주요 경향은 무엇인가?"
➔ Top-k 몇 개의 chunk만 검색해서 전체 corpus의 구조와 관계를 파악하기 어렵다.

기존 Vector RAG만 사용하면 전체 문서의 내용 질문 답변 불가

What Graph-RAG

- "From Local to Global: A Graph-RAG Approach to Query-Focused Summarization", 2024

Document → Chunk → Knowledge Graph

❖ LLM을 이용해 원문에서 entity graph를 만든다

❖ 고정된 Chunk 사용

Dataset	Chunk	Overlap	Chunk 수
Podcast	600 tokens	100 tokens	1,669
News	600 tokens	100 tokens	3,197

```
Entity
- Name
- Type
- Description

Relationship
- Source
- Target
- Description

Claim
- factual statement
```

각 Chunk에서 LLM이 좌측처럼 추출

```
Entity:
NeoChip
Quantum Systems

Relationship:
Quantum Systems
    ↓ owned
NeoChip
```

❖ Triple이라는 단어만 쓰는 것이 아닌 Entity + Relationship

*Triple Extraction: 비구조적인 문장에서 개체와 개체 사이의 관계를 파악해〈주어, 관계, 목적어〉 형태의 구조화된 정보로 바꾸는 NLP 기술

Figure

❖

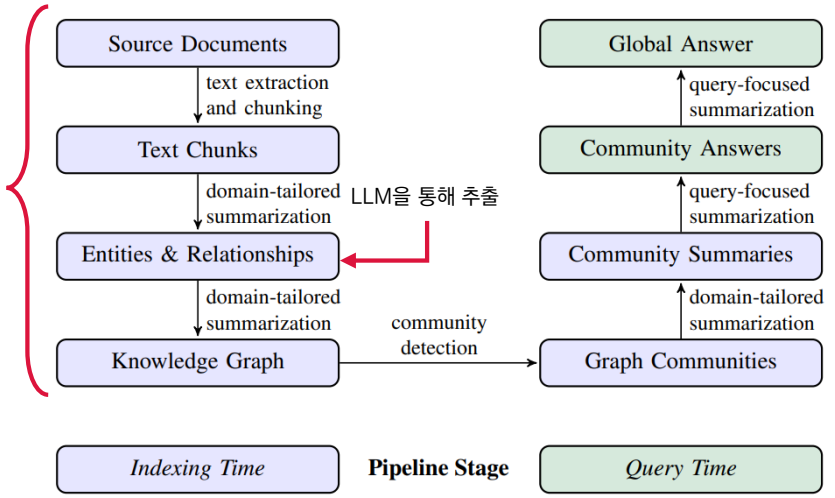


Figure 1: Graph RAG pipeline using an LLM-derived graph index of source document text. This graph index spans nodes (e.g., entities), edges (e.g., relationships), and covariates (e.g., claims) that have been detected, extracted, and summarized by LLM prompts tailored to the domain of the dataset. Community detection (e.g., Leiden, Traag et al., 2019) is used to partition the graph index into groups of elements (nodes, edges, covariates) that the LLM can summarize in parallel at both indexing time and query time. The "global answer" to a given query is produced using a final round of query-focused summarization over all community summaries reporting relevance to that query.

What Graph-RAG

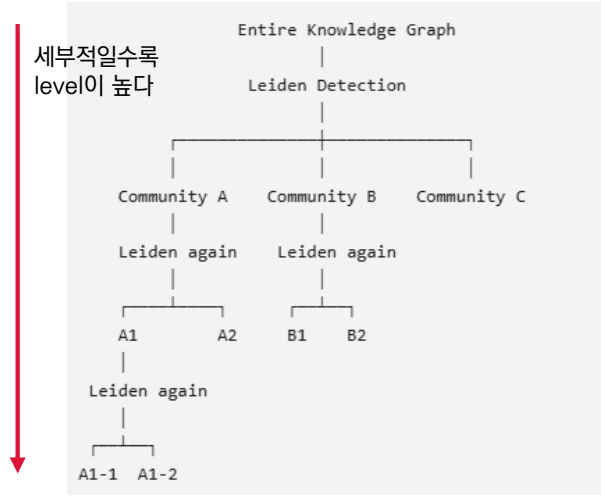
- "From Local to Global: A Graph-RAG Approach to Query-Focused Summarization", 2024

Knowledge Graph → Community Detection

❖ 서로 강하게 연결되어 있는 Node들을 묶어서 Community를 만든다

❖ 여러 community detection algorithm을 사용할 수 있지만, 논문에서 **Leiden community detection**을 사용 그리고 단순히 한 번만 나누는 것이 아니라 **hierarchical하게 재귀적으로 partition**한다

Leiden Community Detection: **그래프에서 서로 연결한 강한 node끼리 그룹으로 묶는 알고리즘**




```
Level 0 : 매우 큰 Community
Level 1 : 조금 더 작은 Community
Level 2 : 더 세부적인 Community
Level 3 : 가장 세부적인 Community
```

라는 계층적 Community 구조를 만든다

➔ 이미 만들어진 graph의 연결 구조 기준으로 Leiden algorithm이 계산

```
Community 1
[A, B, C, D, E, F]

Community 2
[G, H, I, J, K, L]
```

➔ 따라서 Graph-RAG에서는 문서의 텍스트 자체를 다시 clustering하는 것이 아니라 Knowledge Graph의 구조를 clustering하는 것

Figure

❖

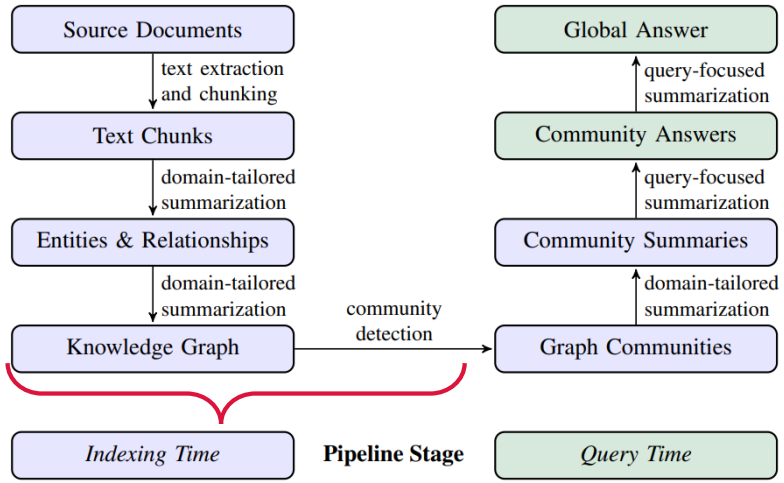


Figure 1: Graph RAG pipeline using an LLM-derived graph index of source document text. This graph index spans nodes (e.g., entities), edges (e.g., relationships), and covariates (e.g., claims) that have been detected, extracted, and summarized by LLM prompts tailored to the domain of the dataset. Community detection (e.g., Leiden, Traag et al., 2019) is used to partition the graph index into groups of elements (nodes, edges, covariates) that the LLM can summarize in parallel at both indexing time and query time. The "global answer" to a given query is produced using a final round of query-focused summarization over all community summaries reporting relevance to that query.

What Graph-RAG

- "From Local to Global: A Graph-RAG Approach to Query-Focused Summarization", 2024

Graph Communities → Community Summaries

```
Community A
  ├─ Quadrotor
  ├─ Rotor
  ├─ Propeller
  ├─ Brushless Motor
  └─ ESC

Community B
  ├─ Battery
  ├─ LiPo
  ├─ Energy Density
  ├─ Voltage
  └─ Flight Time

Community C
  ├─ Payload
  ├─ Camera
  ├─ Sensor
  └─ Surveillance
```

각 Community가 무엇에 관한 그룹인지 LLM이 미리 정하게 한다

```
Graph Communities
        ↓
LLM Summarization
        ↓
Community Summaries
```

```
Community A Summary

"This community describes the propulsion system of
multirotor UAVs, including rotors, propellers,
brushless motors and electronic speed controllers..."
```

Community A를 LLM에 넣으면 대략 위처럼 Community Summary가 생성

Figure

❖

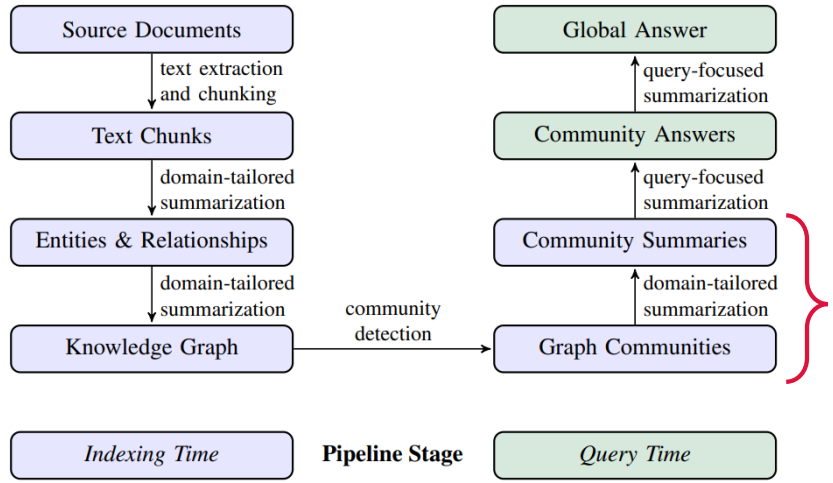


Figure 1: Graph RAG pipeline using an LLM-derived graph index of source document text. This graph index spans nodes (e.g., entities), edges (e.g., relationships), and covariates (e.g., claims) that have been detected, extracted, and summarized by LLM prompts tailored to the domain of the dataset. Community detection (e.g., Leiden, Traag et al., 2019) is used to partition the graph index into groups of elements (nodes, edges, covariates) that the LLM can summarize in parallel at both indexing time and query time. The "global answer" to a given query is produced using a final round of query-focused summarization over all community summaries reporting relevance to that query.

What Graph-RAG

- "From Local to Global: A Graph-RAG Approach to Query-Focused Summarization", 2024

Query Processing – Map & Reduce

❖ 이 논문에선 **Map-Reduce** 방식을 사용

```
Community Summaries
        +
      Query
        ↓
Community Answers
        ↓
Global Answer
```

Map-Reduce 사용 전에 Community Summary를 준비한다.

➔ Random shuffle로 Community Summary를 무작위로 순서를 섞는다

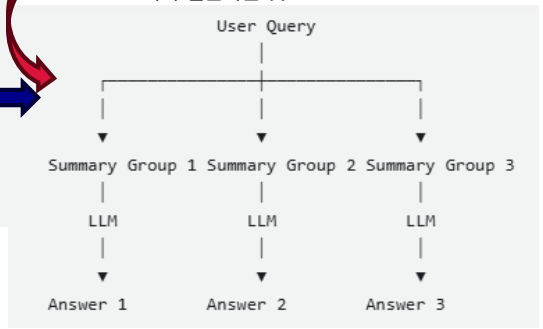
➔ 그리고 Token-size 기준으로 묶는다

➔ 한 쪽으로 몰리는 것을 방지

Before	After shuffle
C1	C17
C2	C2
C3	C83
C4	C41
C5	C6
...	...

Group 1
- C17
- C2
- C83
- ...

Map 단계
➔ 각 community Summary 묶음을 독립적으로 LLM에게 질문하는 것



❖ Map 단계에서 나온 답변에 score를 부여하여 0인 점수는 제거

❖ Score가 높은 점수부터 정렬하여 낮은 점수 제거

```
User Query
+
High-score Intermediate Answers
```

→ Global Answer가 생성

Figure

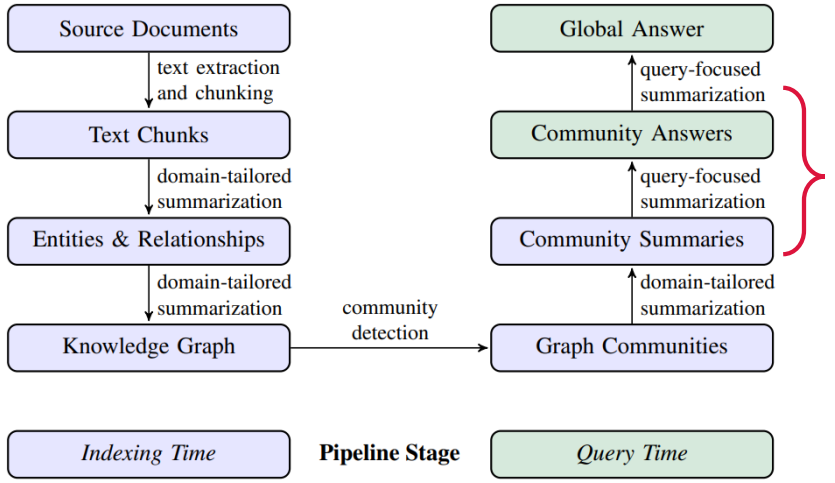


Figure 1: Graph RAG pipeline using an LLM-derived graph index of source document text. This graph index spans nodes (e.g., entities), edges (e.g., relationships), and covariates (e.g., claims) that have been detected, extracted, and summarized by LLM prompts tailored to the domain of the dataset. Community detection (e.g., Leiden, Traag et al., 2019) is used to partition the graph index into groups of elements (nodes, edges, covariates) that the LLM can summarize in parallel at both indexing time and query time. The "global answer" to a given query is produced using a final round of query-focused summarization over all community summaries reporting relevance to that query.

KG-RAG Framework

Framework

- “Biomedical knowledge graph-optimized prompt generation for large language models”, Bioinformatics, 2024

KG-RAG 사용 이유

❖ 논문에서 LLM이 가지고 있는 implicit knowledge만으로는 전문 분야에서 신뢰성 있는 답변을 만들기 어렵기에 KG-RAG 제안

❖ 최신 지식 부족, 출처(provenance) 부족 문제

❖ 이 논문 저자의 생각은

LLM의 implicit knowledge + KG의 explicit knowledge

❖ 이 논문에서는 KG를 설계하지 않고 biomedical KG인 SPOKE 사용

KG-RAG는 LLM의 implicit knowledge에 Knowledge Graph의 구조화된 explicit knowledge를 검색 및 제공하여, hallucination을 줄이고 출처와 근거를 포함한 신뢰성 있는 응답을 생성하기 위한 RAG framework

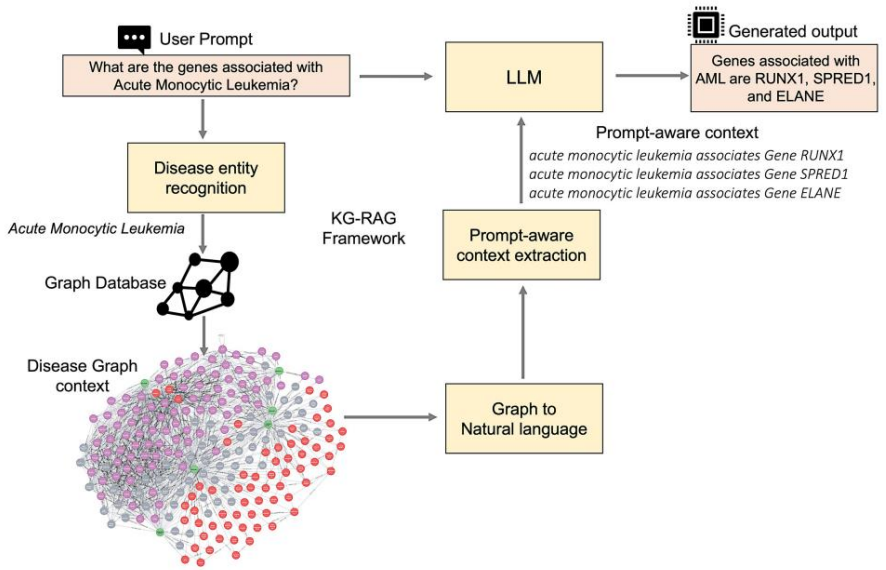


Figure 1. Schema for the Knowledge Graph based Retrieval-Augmented Generation (KG-RAG) Framework. The direction of the arrows indicates the flow of the pipeline in this framework.

KG-RAG Framework

Framework

- “Biomedical knowledge graph-optimized prompt generation for large language models”, Bioinformatics, 2024

Framework

1. 질문 입력
2. 질문으로부터 entity 추출(LLM활용, GPT-3.5-Turbo)
3. Entity를 embedding하여 KG node 찾기

```
"Acute Monocytic Leukemia"

      MiniLM

[0.14, -0.08, 0.27, ..., 0.09]
         384 dimensions
              ↓
      Vector similarity
              ↓
SPOKE disease node matching
```

Chroma Vector DB에 vector 저장

Vector Similarity는 KG 대신 사용되는 것이 아니라, KG에 시작 node를 찾기 위해 사용

4. KG에서 관련 context를 가져오기

```
Disease Node
      ↓
Neighborhood Search
      ↓
S – P – O
S – P – O
S – P – O
```

찾은 Disease Node의 주변 neighbor를 SPOKE REST API를 이용해 가져온다.

5. Prompt-aware Context Pruning

```
Question embedding
       ↓
   cosine similarity

Context 1 – 0.82
Context 2 – 0.74
Context 3 – 0.63
Context 4 – 0.41
Context 5 – 0.21
...
```

질문에 의미적으로 관련된 graph context만 전달

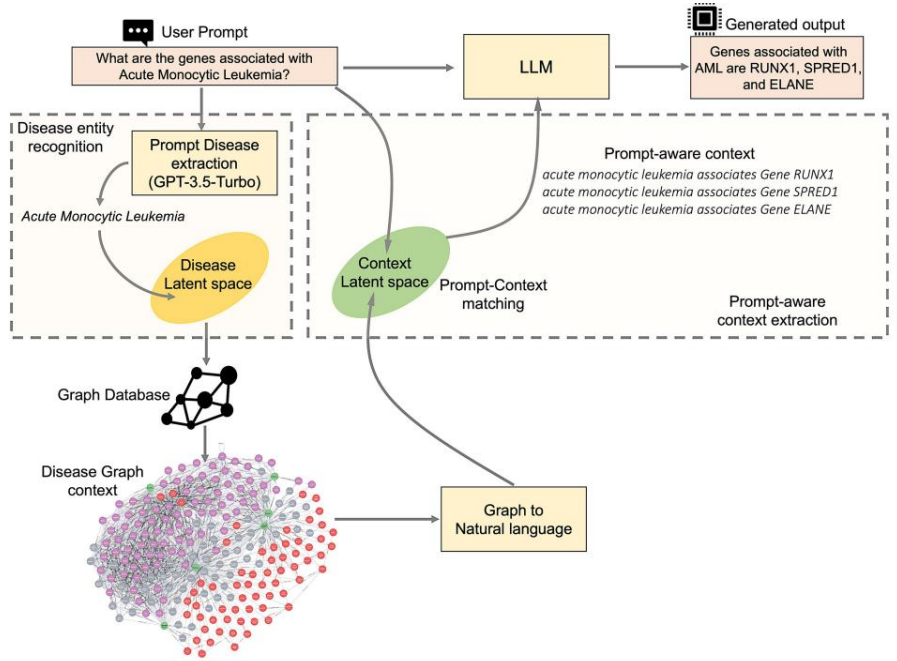


Figure 2. Detailed schema of KG-RAG. Dashed boxes show the details of “Disease entity recognition” and “Prompt-aware context extraction” from the Knowledge Graph.

KG-RAG Framework

KG-RAG Result

- "Biomedical knowledge graph-optimized prompt generation for large language models", Bioinformatics, 2024

결과

❖ 100개의 질문한 결과 LLM 응답 정확성

방법	Retrieval Accuracy	Query perturbation 후
Full-text index	61%	58%
Cypher-RAG	75%	0%
KG-RAG	97%	97%

❖ Token 사용량

Cypher RAG는 사용자의 자연어 질문을 LLM이 그래프 데이터베이스의 쿼리 언어인 Cypher로 변환하고, 이를 실행해 얻은 구조화된 데이터를 다시 LLM에게 컨텍스트로 제공하여 정확한 답변을 생성하는 Graph-RAG 구현 방식

방법	평균 Token
Full-text index	10,590
Cypher-RAG	8,006
KG-RAG	3,693

T/F = True / False, 참/거짓 문제
MCQ = Multiple Choice Question, 객관식 문제

❖ LLM Accuracy(1% = 0.01)

Model	T/F LLM only	T/F KG-RAG	MCQ LLM only	MCQ KG-RAG
Llama-2-13B	0.89	0.94	0.31	0.53
GPT-3.5-Turbo	0.87	0.95	0.63	0.79
GPT-4	0.90	0.95	0.68	0.74

*Context Volume: LLM으로 전달할 수 있는 KG association의 최대 개수
실험 결과 대부분 downstream task에 150, True/False에는 100 사용

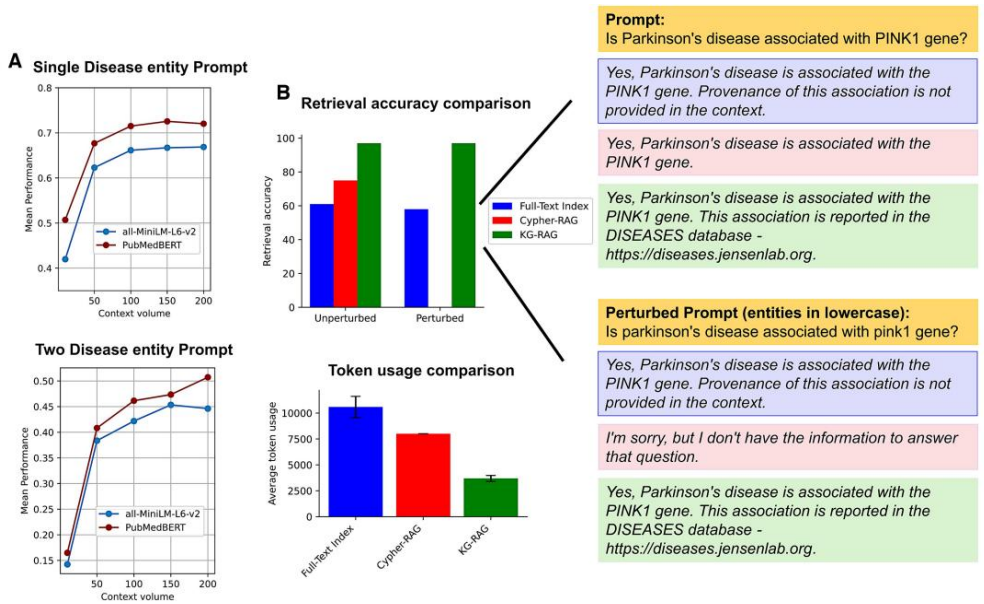


Figure 4. Hyperparameter analysis and RAG comparison. (A) Hyperparameter analysis performance curves using prompts with single (top) and two (below) disease entities mentioned in it. The x-axis denotes the "Context volume" (number of associations from KG) and the y-axis denotes the mean performance (Jaccard similarity) across the prompts. The red curve denotes "PubMedBert" and the blue curve denotes "MiniLM" transformer models. (B) The comparative analysis between KG-RAG, Cypher-RAG, and Full-Text Index using Apache Lucene in terms of retrieval accuracy (top) and token usage (bottom). Insight shows an example where Cypher-RAG fails to retrieve context from the KG when the input prompt is slightly perturbed, but KG-RAG and Full-Text Index remain robust in context retrieval. However, only KG-RAG could extract provenance information linked to the assertion from the context, whereas Full-Text Index could not. It is evident that KG-RAG has significantly lesser token usage when compared to Cypher-RAG and Full-Text Index method (bottom). Error bar in the token utilization bar plot (bottom) represents standard error of the mean (SEM).

Original Chunking + Contextual Retrieval Preprocessing

Context + Chunk

- Introducing Contextual Retrieval

"For an AI model to be useful in specific contexts, it often needs access to background knowledge.", 2024

이 빨간 박스만 먼저 구현하기

기존 RAG와의 차이

❖ Contextual Retrieval 방법: 각 Chunk에 원문 전체에서의 위치와 의미를 설명하는 짧은 문맥을 먼저 붙인 뒤 검색용 데이터로 저장

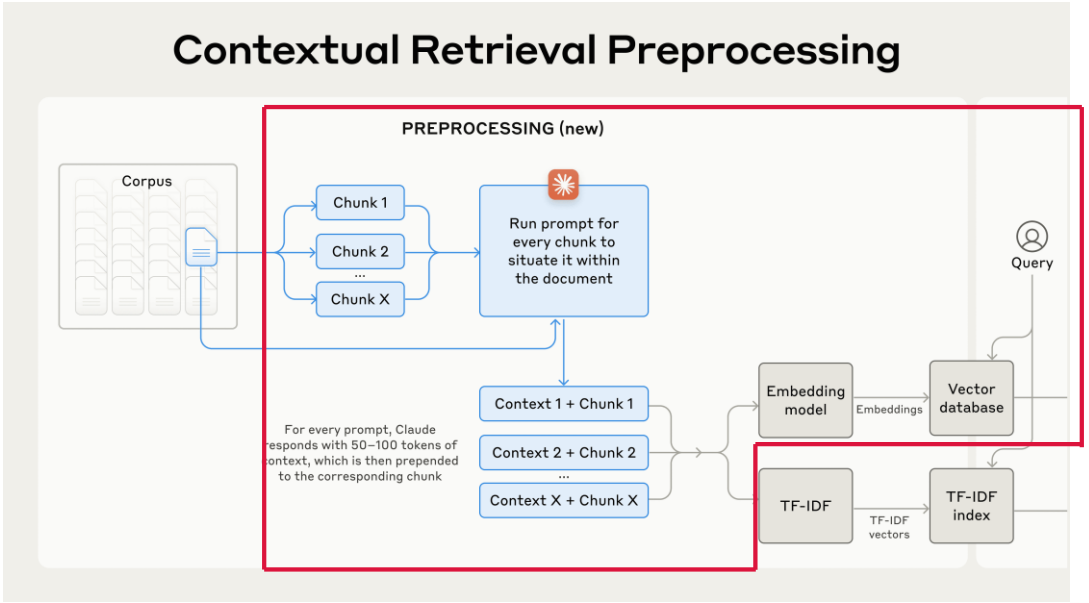
❖ 기존 RAG

Document → Chunking → Embedding → Vector DB → Query → Top-K Retrieval → LLM

❖ Contextual Retrieval

Document → Chunking → **각 Chunk에 문맥 설명 생성** → **[Context + Chunk]** → **Embedding/BM25** → Vector DB → Query → Top-K Retrieval → LLM

figure



Original Chunking + Contextual Retrieval Preprocessing

기존 Chunk와 문맥기반 검색을 통합하여 RAG만 적용한 LLL 응답 workflow

- Structure-Aware Semantic Contextual Chunking
- Paper table

❖ Table 및 코드에 적용할 모델

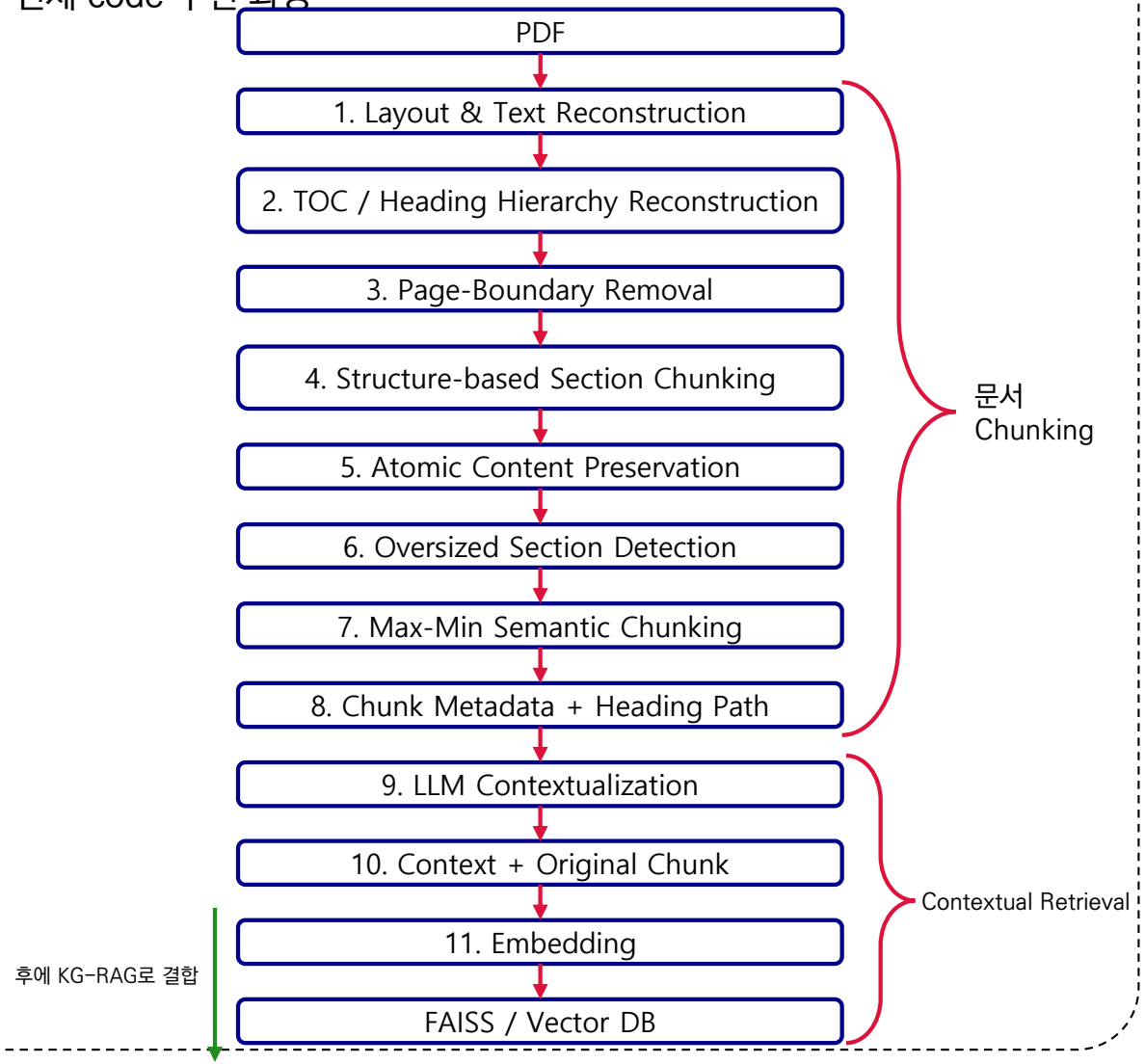
우선순위	논문	핵심적으로 가져올 부분	현재 확인 인용
1	Guo et al., 2025, Enhancing visual-LLM for construction site safety compliance via prompt engineering and Bi-stage retrieval-augmented generation, Elsevier	Hierarchical Content-Aware Chunking, 제목/Chapter/조항 구조 보존	ScienceDirect Cited by 21
2	Kiss et al, 2025, Max–Min semantic chunking of documents for RAG application, Spriner Nature	Max-Min Semantic Chunking. Section 내부 의미 경계 판단	Springer 11
3	Plonka et al, 2025, A comparative evaluation of the effectiveness of document splitters for large language models in legal contexts, Elsevier	Splitter 방법 비교 및 document formatting 중요성	Scopus 13, GS 24
4	Moreno-Cediel et al, 2026, Optimising retrieval performance in RAG systems: A new growing window semantic chunking strategy to address weak semantic boundaries, Elsevier	Growing Window, 애매한 semantic boundary 보완	최신 논문
보조	"Evaluating RAG Accuracy Enhancement via Page Merging for Non-Ideal Documents," Procedia Computer Science, Elsevier, 2025	물리적 page break 때문에 문맥을 끊지 않는 것	최신 연구

Model	다국어	최대 입력	Embedding 차원	장점
BGE-M3	100+	8192 tokens	1024	다국어 + Dense/Sparse/Multi-vector 검색
Qwen3-Embedding-0.6B	**100+**	32K	최대 1024	최신 계열, instruction-aware, cross-lingual
Qwen3-Embedding-4B	**100+**	32K	최대 2560	높은 표현력, 복잡한 의미 구분
Jina Embeddings v3	94+	8192	최대 1024	Similarity 전용 모드, 한국어 포함
Multilingual-e5-large-instruct	94+	-	1024	검증된 다국어 retrieval 모델

앞으로 이 모델로 Embedding

LLM은 Mistral 7B에서 Qwen3 8B로 변경, Mistral이 언어 능력이 너무 떨어짐

❖ 전체 code 구현 과정



Original Chunking + Contextual Retrieval Preprocessing

기존 Chunk를 고정 Chunking 방식이 아닌 유동적 Chunking 방식을 적용하는 과정

① Layout & Text Reconstruction

➔ 이 단계의 목적은 단순한 문자열로 읽는 것이 아니라, text / heading / table / figure / list / equation과 같은 요소를 순서대로 복원

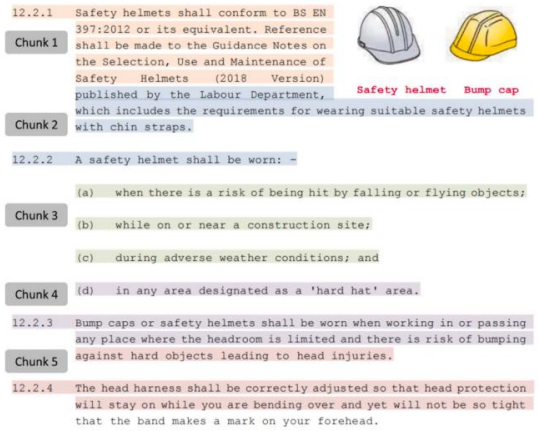


Fig. 4. Example of Traditional Chunking.

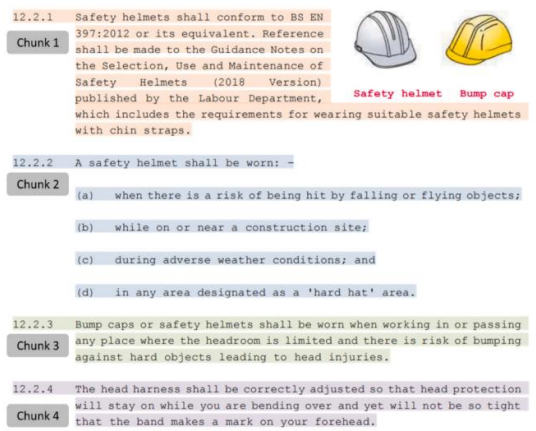


Fig. 5. Example of Content-aware Chunking.

Guo 논문과 구현 코드의 차이점

	Guo et al.	현재 코드
PDF Parser	Apache Tika	PyMuPDF
중간 형식	HTML	직접 PDF 구조 사용
PDF TOC	핵심 요소 아님	추가 사용
번호 3 / 3.1 / 3.1.1	구조 판단	추가 활용
Header/Footer	제거	반복 위치 기반 제거

② TOC / Heading Hierarchy Reconstruction – 대제목, 소제목 구조 복원

TOC: Table Of Contents(목차), level은 오름차순으로 대제목, 소제목 순으로 연결

예로

```
3. Scaling Law Derivation
    3.1 Class I Multirotor
    3.2 Class II Multirotor
4. Heavy-Lift Case Study
```

hierarchy로 만든다

```
Document
├ 3. Scaling Law Derivation
|  ├ 3.1 Class I Multirotor
|  └ 3.2 Class II Multirotor
└ 4. Heavy-Lift Case Study
```

➔ TOC가 없거나 불완전한 코드가 추가로 1, 1.1, 1.1.1 같은 numbering, font size, bold, ALL CAPS, 문장 종결 형태를 이용해 heading 여부를 추정

➔ TOC가 있는 경우에는 해당 페이지 ±1 안에서 문자열 similarity가 0.90 이상이면 TOC heading으로 매칭

heading을 발견할 때마다 리스트에 추가

Heading을 찾으면 heading_stack을 사용, 예로

```
Level 1 = 3. Scaling Law Derivation
Level 2 = 3.2 Class II Multirotor
```

우측과 같이 가지게 된다

```
heading_path =
[
    "3. Scaling Law Derivation",
    "3.2 Class II Multirotor"
]
```

핵심 원칙은

Document Structure → Chunk Boundary Candidate

Original Chunking + Contextual Retrieval Preprocessing

Context-Chunk Methodology

③ Page-Boundary Removal – 페이지가 바뀐다고 내용을 자르지 않음

page로 분리된 text 하나로 복원

```
Page 7
Battery capacity increases with vehicle

Page 8
size and therefore affects MTOW.
```

→

```
Battery capacity increases with vehicle
size and therefore affects MTOW.
```

➔ heading_path이고, 둘 다 text이며 atomic content가 아니고, 앞 문장이 완전히 종료되지 않았거나 다음 page의 첫 text가 continuation 형태일 때 합친다

➔ 이 아이디어의 학술적 근거는 “Evaluating RAG Accuracy Enhancement via Page Merging for Non-Ideal Documents,” Procedia Computer Science, Elsevier, 2025

➔ 해당 연구는 page break 때문에 여러 페이지에 걸친 table/figure 등의 관계가 깨져 RAG accuracy가 떨어질 수 있음을 지적하고 **layout-aware page merging을 제안**

➔ 그러므로

Cross-page Text Continuity

Metadata: **Data about Data로 데이터에 대한 데이터라는 의미**, 예로 data가 있으면 이 data가 어떤 정보를 가진 data인지에 대한 이야기

④ Structure-based Section Chunking – Heading Path를 절대 경계로 사용

➔ heading 자체를 independent content chunk로 넣지 않고 metadata 역할로 사용

➔ 동일한 full heading path를 가지고 연속 요소들을 하나의 section으로 묶는다

```
3.1 Class I

Paragraph A
Paragraph B

3.2 Class II

Paragraph C
Paragraph D
```

이면,

```
Section A
heading_path = 3 > 3.1
Paragraph A + B

Section B
heading_path = 3 > 3.2
Paragraph C + D
```

으로 처리

➔ 따라서 서로 다른 section의 내용을 하나의 chunk에 넣지 않는다

➔ 따라서 가장 중요한 Hard Constraint는:

$$\text{HeadingPath}_A \neq \text{HeadingPath}_B \implies \text{SameChunk 불가}$$

➔ 이 부분도 Guo et al. 논문의 hierarchical/content-aware chunking 원리를 강하게 반영

Original Chunking + Contextual Retrieval Preprocessing

Context-Chunk Methodology

⑤ Atomic Content Preservation –Table/Figure/List/Equation보호

➔ Table, Figure, List, Equation을 atomic element로 간주

예로 글이 paragraph A – Table 3 – Paragraph B 순이라면

```
Paragraph A

Table 3

Paragraph B
```



```
Chunk 1 = Paragraph A
Chunk 2 = Table 3
Chunk 3 = Paragraph B
```

➔ Table을 만나는 순간 text buffer를 먼저 종료하고 table을 하나의 독립 chunk로 만들기 때문에 Max-Min이 table을 가로질러 chunk를 만들 수 없다.

➔ 이 단계는 Guo의 context-aware 구조 보존 원칙에서 영향을 받았지만, **Table/Figure/List/Equation을 atomic hard boundary로 두는 정확한 구현은 현재 연구에 맞게 추가 자체 설계**

Atomic Boundary > Semantic Similarity

⑥ Oversized Section Detection – Max-Min을 사용할지 결정

➔ 현재 코드에서는 아래와 같이 Max-tokens 제한

```
OVERSIZED_SECTION_TOKENS = 600
HARD_MAX_CHUNK_TOKENS = 800
```

$T_{\text{section}} \leq 600$ 이면 `Section 전체 → 하나의 Chunk`

반대로

$T_{\text{section}} > 600$ 이면

```
Section
  ↓
Sentence Split
  ↓
Max-Min
```

을 실행

➔ 실제 코드도 `n_tokens <= OVERSIZED_SECTION_TOKENS` 이면 그대로 chunk를 만들고, 초과하면 `self.maxmin.chunk(text)` 를 실행한다

➔ Kiss 논문에서 정한 값이 아닌 현재 시스템에서 정한 engineering safety parameter 600, 800(언제든지 변경 가능한 값)

$600 =$ 언제 Max – Min을 사용할까?

$800 =$ Max – Min 중 얼마나 커지는 것을 허용할까?

➔ 800은 모든 종류의 chunk에 대한 절대적 최대값은 아니다. ex) 큰 table은 atomic chunk로 따로 저장되므로 800 tokens를 넘을 수도 있다.

Original Chunking + Contextual Retrieval Preprocessing

Context-Chunk Methodology

⑦ Max-Min Semantic Chunking –현재 코드에서 논문을 가장 직접적으로 구현한 부분

➔ Kiss 논문은 chunk을 **dynamic/temporal clustering** 문제로 정의, 즉 문서의 순서를 유지하면서 연속 sentence를 embedding similarity로 묶는다

먼저 각 sentence:

$$s_1, s_2, \cdots, s_n \quad \text{를 embedding한다}$$

$$\rightarrow E(s_i)$$

논문에선 BGE-base-en-1.5를 사용하고, 본 코드에선 BGE-M3 사용

현재 Chunk C 내부의 가장 낮은 sentence similarity를:

$$\min_{\text{sim}}(C) = \min_{s,t \in C} \frac{E(s)^T E(t)}{\|E(s)\| \|E(t)\|}$$

로 계산

그리고 새 sentence s_k 와 기존 chunk 중 가까운 sentence similarity를:

$$\max_{\text{sim}}(s_k, C) = \max_{s \in C} \frac{E(s_k)^T E(t)}{\|E(s_k)\| \|E(t)\|}$$

로 계산

Threshold는:
➔ hard_thr = hard threshold = 절대로 이것보다 낮아지지 않게 최소 기준
➔ 즉 새 sentence를 기존 chunk에 넣을 때 너무 낮은 similarity까지 허용하지 않도록 최소 바닥 값을 만들어 놓은 것

$$thr(C) = \max\{c \cdot \text{min_sim}(C) \cdot \sigma(|C|), hard_{thr}\}$$

이고

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

따라서:

$$\max_{sim}(s_k, C) > thr(C)$$

이면 **기존 chunk에 추가하고, 그렇지 않으면 새로운 chunk를 시작**

```
min_sim = ...
max_sim = ...
threshold = max(
    c * min_sim * sigmoid(len(current)),
    hard_thr
)

add = max_sim > threshold
```

로 코드에서 구현하고 있고, 첫 sentence 2개는 별도 initialization rule:

$$init_{const} \cdot sim(s_{k-1}, s_k) > hard_{thr}$$

을 사용

현재 기본 parameter는 논문의 추천 초기 값

$$hard_{thr} = 0.6 \qquad init_{const} = 1.5 \qquad c = 0.9$$

를 그대로 사용 ➔ 초기값은 Kiss논문에서 sensitivity analysis와 cross-validation 결과를 바탕으로 초기 권장값으로 제시

Original Chunking + Contextual Retrieval Preprocessing

Context-Chunk Methodology

⑧ Chunk Metadata + Heading Path – 잘린 Chunk가 어디에서 왔는지 보존

➔Max-Min으로 text를 잘 나눴다고 끝이 아니다

각 Chunk에 아래와 같이 저장

```
chunk_id
doc_id
source_file
document_title
heading_path
page_start
page_end
content_type
original_text
section_key
order
token_count
```

예를 들면

```
Document:
Sizing of Multirotor Aerial Vehicle

Heading Path:
3. Scaling Law Derivation
> 3.2 Class II Multirotor

Pages:
8-9

Content Type:
text
```

➔ Guo et al.의 hierarchical context-aware 방법과 연결되는 부분은 **본문만 저장하지 않고 title/chapter/source 같은 structural context를 유지한다는 원칙**

➔ 다만 현재 코드에서는 안정적인 chunk_id hashing 방식:

$$ID = hash(doc + section + page + type + text)$$

은 현재 시스템에서 추가한 자체 구현

⑨ LLM Contextualization

```
system = (
    "You create short retrieval context for RAG chunks. "
    "Use ONLY the supplied source information. Do not invent facts. "
    "Do not rewrite the original chunk. Explain what the chunk is about, "
    "what entity/topic it refers to, and where it sits in the document when useful. "
    "Return only 1-3 concise sentences, preferably 50-100 tokens. "
    "Write in the primary language of the source chunk (usually English)."
)
```

➔ 이 Chunk가 Retrieval될 때 무슨 내용인지 이해할 수 있도록 짧은 설명을 만든다

➔ 수식으로 요약하자면

$$C_{context} = \text{Topic} + \text{Entity} + \text{DocumentPosition}$$

```
Document Title
+
TOC
+
Heading Path
+
Parent Section Context
+
Chunk Type
+
Original Chunk
```

현재 prompt는 Qwen에게 **현재 chunk가 무엇에 관한지, 어떤 entity/topic을 가리키는지, 문서에서 어디에 위치하는지** 설명하고 1~3문장, 가능하면 50~100tokens로 만들도록 한다

Original Chunking + Contextual Retrieval Preprocessing

Intelligent Design for Engineering Applications Lab

Context-Chunk Methodology

⑩ Context + Original Chunk

➔ Debug code를 활용하여 실제 어떻게 Chunking되었는지 확인

chunk의 고유 ID

어느 pdf인지

현재 Text 묶음을 그대로 하나의 Chunk로 만들었다

이유

Original Chunk

Qwen이 생성한 context

Qwen이 Context를 만들 때 참고한 상위 section의 내용

최종 context + chunk로 이 전체 문자열이 embedding 모델로 들어간다

```
## Chunk 2: `394b44183d04_de78b59dd29a`

- File: `book₩멀티로더 Historical Data₩1-s2.0-S2590123026024254-main.pdf`
- Pages: `1-1`
- Heading path: `[Unsectioned]`
- Content type: `text`
- Tokens: `8`
- Creation method: `WHOLE_TEXT_BUFFER`

### Why was this chunk created?

WHOLE_TEXT_BUFFER: token_count=8 <= OVERSIZED_SECTION_TOKENS=600; Max-Min was not invoked.

### Original chunk

```text
Contents lists available at ScienceDirect
```

Qwen-generated retrieval context

```text
The chunk indicates that the document's contents are available through ScienceDirect, a platform for scientific research. It serves as an introductory note about the document's accessibility. This information is presented at the beginning of the document.
```

Context input: Parent section context

```text
[figure] [Figure/Image block on page 1; visual pixels not interpreted]

[text] Contents lists available at ScienceDirect

[figure] [Figure/Image block on page 1; visual pixels not interpreted]
```

Final Context + Original Chunk input to embedding

```text
Document: Sizing of Multirotor Aerial Vehicle: A Statistical Approach
Section: [Unsectioned]
Content type: text
Context: The chunk indicates that the document's contents are available through ScienceDirect, a platform for scientific research. It serves as an introductory note about the document's accessibility. This information is presented at the beginning of the document.
Content: Contents lists available at ScienceDirect
```

```

⑩이후 KG-RAG를 활용하지 않고 RAG만 사용했을 때 LLM 응답

1. 질문

```
Question> 드론을 1종으로 설계하려고 하는데 어떻게 하면 좋을까요?

[1/4] Encoding query with BGE-M3 and retrieving from FAISS...
2026-09-20 11:45:43,289 | INFO | Loading embedding model BAAI/bge-m3 on cpu ...
```

2. 관련 vector Similarity로 출력된 Context + Chunk

```
--- Retrieved chunks ---
[S1] score=0.5616 | book\멀티로더 Historical Data\1-s2.0-S2590123026024254-main.pdf | p.1-2 | Sizing of Multirotor Aerial Vehicle: A Statistical Approach > 1. Introduction
    [11] proposed dimensional based scaling laws and sim corresponding to mission priorities. To account for these aspects, de ilarity models for multirotor drone
    components such as motors, ESCs, signers must balance coupled factors such as propulsion efficiency, batteries, propellers, and structure and validated against
    platforms * Corresponding autho
[S2] score=0.5426 | book\멀티로더 Historical Data\1-s2.0-S2590123026024254-main.pdf | p.16-17 | 2 - 2-cos (0.1t) > References
    Mahony, Design principles of large quadrotors for practical Menon, Development of Agricultural Drone for Crop Monitoring and Disease applications, in: 2009 IEEE
    Int.
[S3] score=0.5372 | book\멀티로더 Historical Data\1-s2.0-S2590123026024254-main.pdf | p.1-2 | Sizing of Multirotor Aerial Vehicle: A Statistical Approach > 1. Introduction
    Bouabdallah et al. [3] demonstrated a applications, such as agricultural management, aerial imaging, geo propulsion-group design methodology for sub-kilogram VTOL
    plat spatial mapping, logistics, disaster relief, military reconnaissance, and forms, applying combined momentum and blade element theory to warfare operations.
[S4] score=0.5323 | book\멀티로더 Historical Data\1-s2.0-S2590123026024254-main.pdf | p.14-15 | 2 - 2-cos (0.1t) > RD
    The JMRRC has a different mission priorities, propulsion configurations, and design strategies. The framework gives a reliable first-order estimate corre
    incuLcating dynamic simulation into the design process of the sponding to common design trends, while deviations occur when designs multirotor. follow different
    objectives within the broader desig
[S5] score=0.5242 | book\멀티로더 Historical Data\1-s2.0-S2590123026024254-main.pdf | p.16-17 | 2 - 2-cos (0.1t) > References
    Abdelkefi, Classifications, applications, and design challenges 106773, https://doi.org/10.1016/j.ast.2021.106773. of drones: A review, Prog Aerosp Sci 91 (2017)
    99-131, https://doi.org/10.1016/j. [5] Raymer Daniel, Aircraft Design: A Conceptual Approach, Sixth Edition, American paerosci.2017.04.003. Institute of
    Aeronautics and Astronautics, Inc.,
[S6] score=0.5232 | book\멀티로더 Historical Data\1-s2.0-S2590123026024254-main.pdf | p.2-3 | 4. To bring out a scaling and sizing methodology for multirotors > 2. Survey
 & Database Formation
    Filippone's elec Drones (TD), and Racing Drones (RD). Consequently, the scaling laws tric multirotor database was used as initial reference [31].
2026-09-20 11:45:50,267 | INFO | Unloading embedding model from memory ...

[2/4] Building RAG context...
      ✓ Context ready

[3/4] Loading Qwen3:8b...
      ✓ Ready

[4/4] Generating answer (streaming)...
```

3. LLM 응답

```
드론을 1종으로 설계하려면 다음과 같은 단계를 고려하는 것이 좋습니다:

1. **미션 우선순위 설정**: 드론의 주요 목적(예: 농업 모니터링, 물류, 재난 구조 등)을 명확히 정의하고, 이에 따라 차량의 크기, 무게, 비행 시간 등을 결정해야 합니다. 이는 [S1]에서 언급된 것처럼, 차량의 설계에 영향을 미치는 다양한 요소(예: 추진 효율, 배터리, 프로펠러, 구조 등)를 균형 있게 고려해야 합니다.

2. **비행기 유형 및 규모 선택**: 드론의 유형(예: 쿼드로터, 바트로터 등)과 규모를 결정해야 합니다. [S3]에서는 Bouabdallah 등이 제시한 추진 그룹 설계 방법론을 통해 다양한 용도(예: 농업 관리, 항공 촬영, 물류 등)에 맞는 드론을 설계할 수 있음을 설명하고 있습니다.

3. **비행 성능 및 에너지 요구 사항 분석**: 드론의 비행 시간, 최대 속도, 하중 용량 등을 고려하여 배터리, 프로펠러, 모터 등의 성능을 평가해야 합니다. [S4]에서는 JMRRC의 설계 프레임워크를 통해 초기 추정치를 제공하며, 이는 비행 성능과 에너지 요구 사항을 고려한 설계에 도움이 됩니다.

4. **비행 제어 및 안정성 설계**: 드론의 비행 제어 시스템과 안정성을 고려해야 합니다. [S2]에서는 Mahony과 Menon의 연구를 통해 대형 쿼드로터의 설계 원칙을 설명하고 있으며, 이는 비행 제어 및 안정성 설계에 중요한 참고 자료가 됩니다.

5. **데이터베이스 및 스케일링 법칙 활용**: [S6]에서는 Filippone의 전기 드론 및 레이싱 드론 데이터베이스를 사용하여 스케일링 법칙을 도출하는 방법을 제시하고 있습니다. 이는 드론 설계 시 참고할 수 있는 초기 데이터베이스로 활용할 수 있습니다.

6. **실제 플랫폼과의 검증**: 설계된 드론은 실제 플랫폼과 비교하여 검증해야 합니다. [S1]에서는 설계 요소(모터, ESC, 배터리, 프로펠러, 구조 등)를 실제 플랫폼과 비교하여 검증하는 것이 중요하다고 강조하고 있습니다.

이러한 단계를 통해 드론 설계를 체계적으로 접근할 수 있으며, 설계 과정에서 발생하는 다양한 요소를 고려하여 최적의 1종 드론을 설계할 수 있습니다.
```

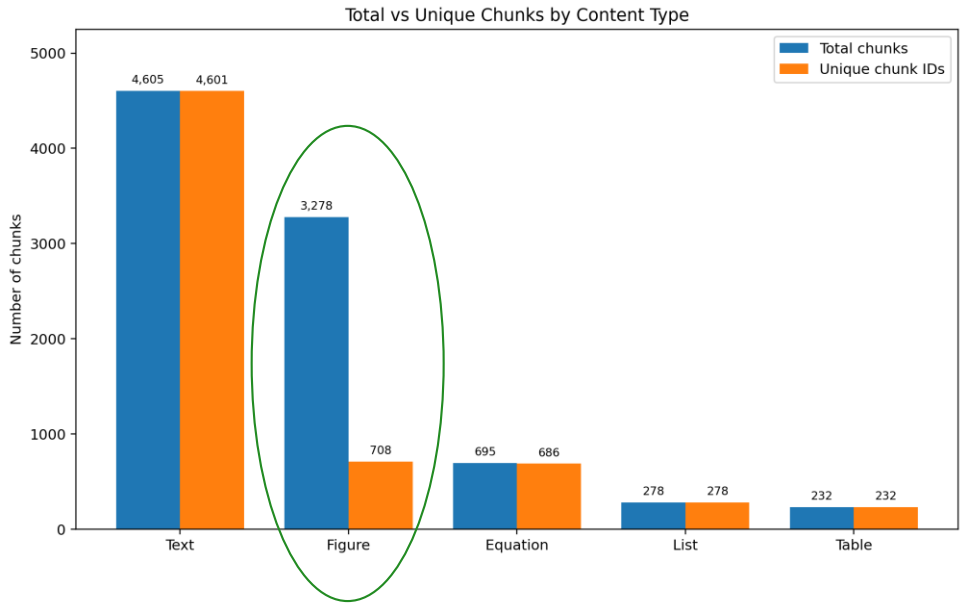
Original Chunking + Contextual Retrieval Preprocessing

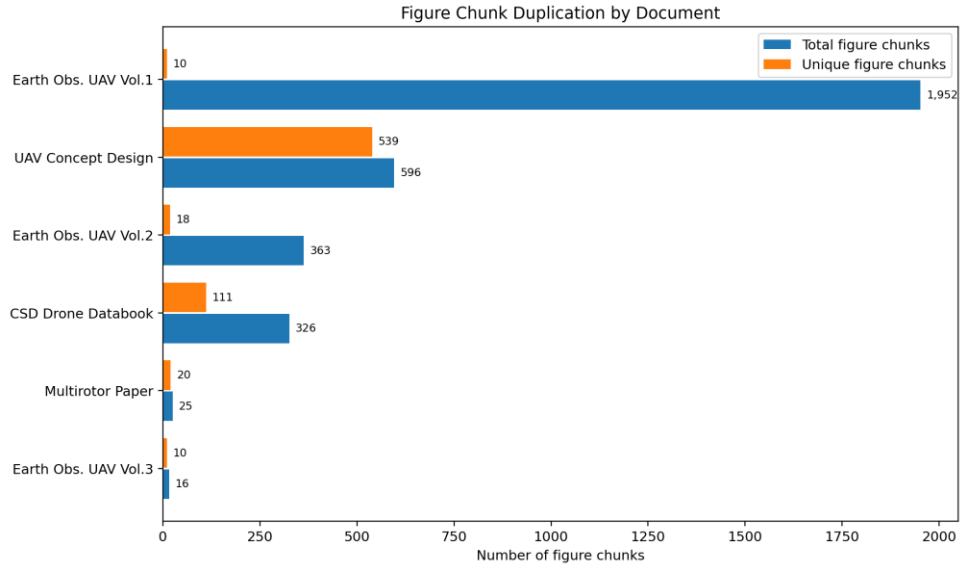
Code 결과

항목	결과
입력 PDF	6 개
생성 Chunk	9,088 개
고유 Chunk ID	6,505 개
중복 Chunk	2,583 개
중복 비율	28.4%
Embedding	BGE-M3
Embedding dimension	1,024
Context 생성 LLM	Qwen3:8b
FAISS Vector	9,088 개
전체 Build 시간	4, 621 sec ≈ 77min (Colab A100 GPU)
Median chunk length	18 tokens
20 tokens 이하	5,609 개 (61.7%)

PDF 실제 정보 → Embedding만 들어가는 것이 아니라
이미 내용 모름 → LLM이 주변 문맥으로 추측 → 그 추측을 Embedding

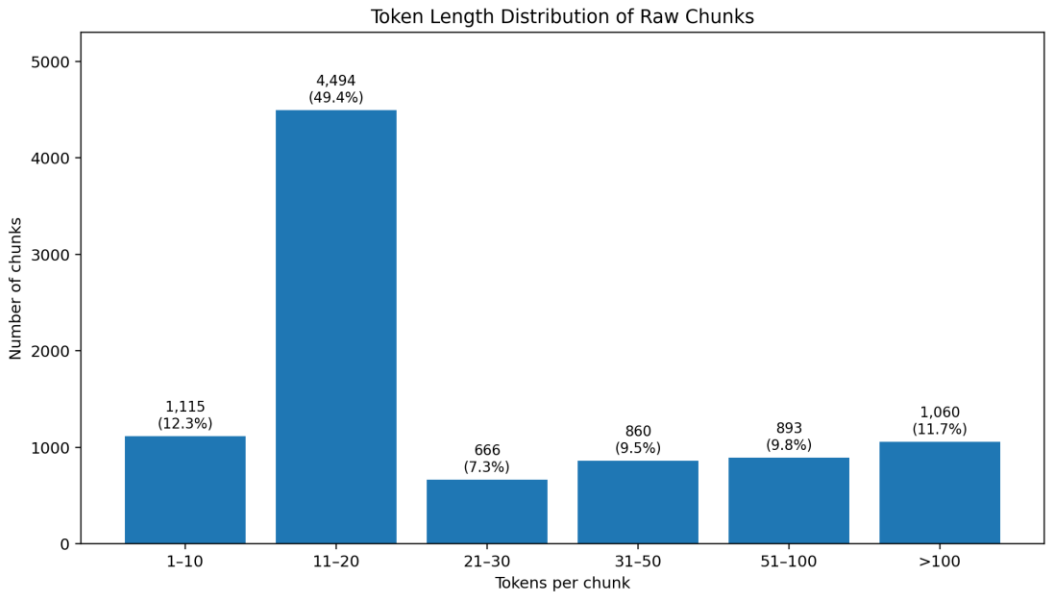
➔ 즉 Figure 존재만 감지하고 주변 문맥을 보고 LLM이 내용 추정 후 context 생성





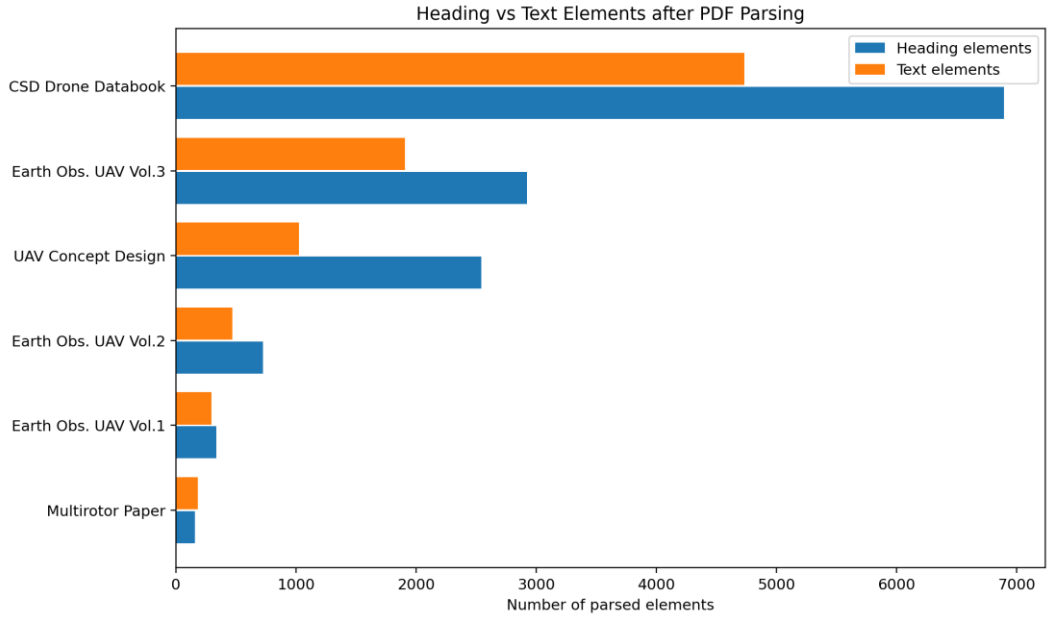
Original Chunking + Contextual Retrieval Preprocessing

Code 결과



➔ 현재 사용하고 있는 방법은 **document structure를 최대한 보조하면서 semantic/structural chunk를 생성하는 방식**

➔ 실제 결과에서는 너무 짧은 구조 요소들이 각각 독립 chunk로 남은 경우가 많다



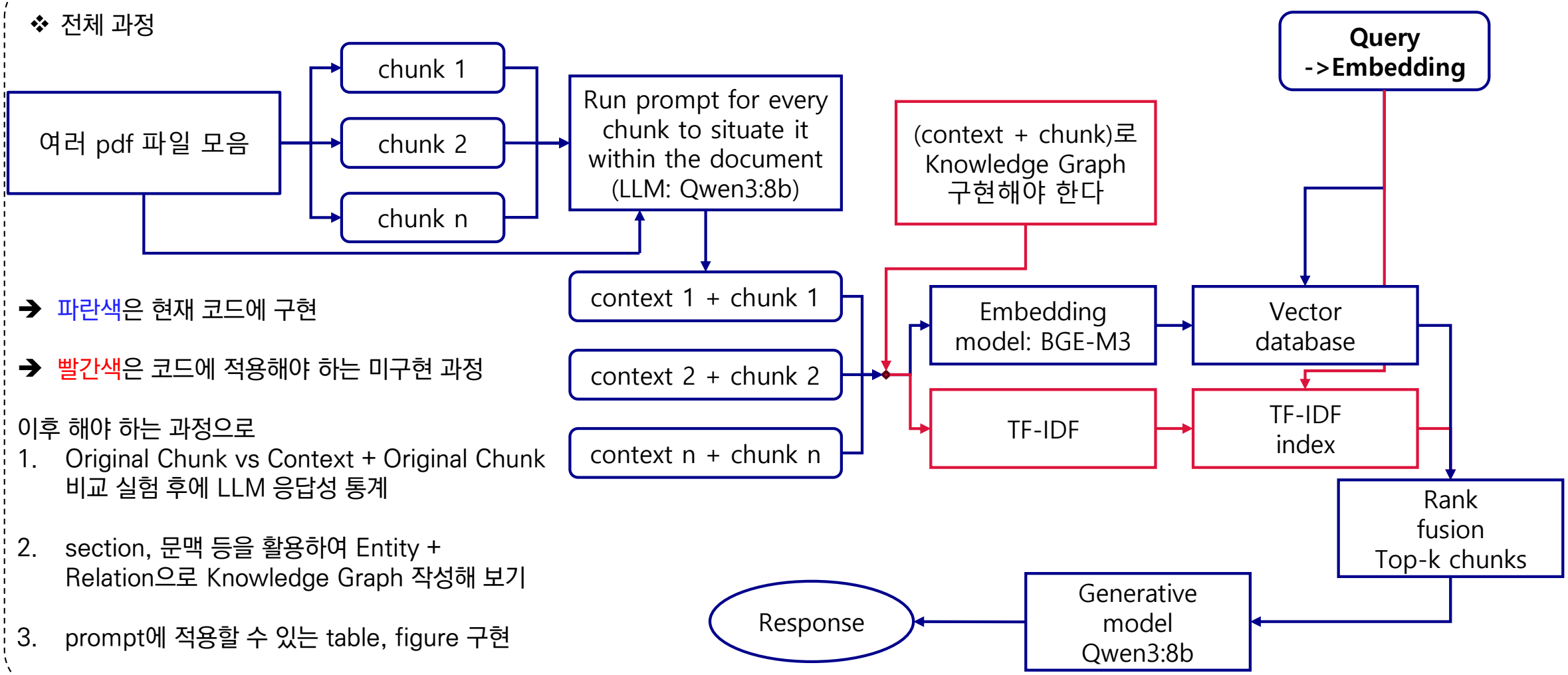
➔ PDF 구조 인식 단계가 문서 내용을 어떻게 분류했는지 확인하기 위한 그래프

➔ 문제는 일부 문서에서 Heading의 개수가 지나치게 많다는 점

➔ 예로 **CSD Drone Databook에서 Heading이 약 6,893개, Text가 약 4,733개로 나타났고**, 일반적인 기술문서라면 제목보다 본문이 훨씬 많은 경우가 자연스럽다. 하지만 이 결과에선 Heading이 더 많다.

구현한 전체 과정과 이후 해야 하는 작업

❖ 전체 과정



➔ 파란색은 현재 코드에 구현

➔ 빨간색은 코드에 적용해야 하는 미구현 과정

이후 해야 하는 과정으로

1. Original Chunk vs Context + Original Chunk 비교 실험 후에 LLM 응답성 통계

2. section, 문맥 등을 활용하여 Entity + Relation으로 Knowledge Graph 작성해 보기

3. prompt에 적용할 수 있는 table, figure 구현

Any Questions?

이름 신 동 우

소속 한양대학교 기계공학과

이메일 sunwoongy@hanyang.ac.kr

Intelligent Design for Engineering Applications Lab

